

Progress Tracking & Certificate Issuance

1. Feature Name

Progress Tracking & Certificate Issuance

2. Purpose

- **Problem Solved:** Learners need a way to record which modules they have finished and to know how far along they are in a learning path.
 - **Why It Exists:** Tracking module completion enables prerequisite gating, progress visualization, dashboards, and analytics. When a user completes every module of a path, the platform automatically issues a certificate as proof of completion.
 - **Role in Application:** This feature produces the completion state consumed by the Learning Path Engine (unlock logic), Dashboard, Analytics, and Certificates. It is the record of academic achievement in the platform.
-

3. What the User Can Do

Student

- View their own per-path progress (completed modules, total modules, percentage, unlock state).
- See whether they are certificate-eligible for a path (100% complete).
- List their issued certificates.

Instructor / Admin

- Mark a module as complete on behalf of a student (subject to prerequisite checks).
 - Module completion may also occur automatically when a student passes an assigned quiz.
-

4. Feature Workflow

[Student finishes **module** content]

→ (quiz pass **OR** instructor/admin calls) → MarkComplete / MarkModuleCompleteFromQuizAsync

- ProgressService verifies prerequisite modules **are** completed
- Progress **row** created/updated (IsCompleted = **true**, CompletedAt = now)
- MODULE_COMPLETED audit logged
- TryIssueCertificateAsync: if **all** modules **of** the path complete **and** no certificate **exists** → c

[Student views progress]

- **GET** /api/v1/progress
- **For each** path touched, BuildPathSummaryInternalAsync computes:
 - completed **set** → **per-module** isUnlocked (deps $\subseteq$ completed) → percentage
- **Returns** PathProgressSummaryDto list

5. How It Works Internally

Mark Complete (with prerequisite enforcement)

ProgressService.MarkCompleteAsync(dto, userId):

1. Loads the module with its Dependencies and LearningPath.
2. Collects depIds = module.Dependencies.Select(d => d.DependsOnModuleId).
3. If the module has dependencies, counts the user's completed progress rows that match those dependency ids.
4. If completedDepCount < depIds.Count → throws ArgumentException("Complete all prerequisite modules first."). Completion is rejected.
5. If a Progress row exists and is already completed → throws "Module already completed."
6. Otherwise updates it (or creates it) with IsCompleted = true, CompletedAt = DateTime.UtcNow.
7. Saves, logs MODULE_COMPLETED audit, then calls TryIssueCertificateAsync(pathId, userId).

Quiz-Driven Completion

MarkModuleCompleteFromQuizAsync(userId, moduleId) is invoked by the Attempt Service when a student passes an assigned quiz. It performs the same upsert without prerequisite re-checks (prerequisites were already enforced by the quiz flow) and also triggers certificate issuance.

Progress Summary Calculation

BuildPathSummaryInternalAsync(pathId, userId):

1. Loads the path with modules and dependencies.
2. Loads the user's completed progress ids for those modules as a dictionary moduleId → CompletedAt.

3. For each module: `isUnlocked = dependencyIds.All(d => completedIds.ContainsKey(d));`
`isCompleted = completedIds.ContainsKey(module.Id);`
4. `completed = count of completed modules;` `total = total modules.`
5. `ProgressPercent = total > 0 ? (int)Math.Round((double)completed / total * 100) : 0.`
6. `IsCertificateEligible = (ProgressPercent == 100).`

`GetUserProgressAsync(userId)` derives the set of enrolled paths (`Distinct module LearningPathId` from the user's progress rows) and builds one summary per path.

Certificate Auto-Issuance

`TryIssueCertificateAsync(pathId, userId):`

1. Loads the path with its modules; returns if the path is null or has no modules.
2. Counts the user's completed progress rows for this path's module ids.
3. If `completedCount < path.Modules.Count` → not eligible, returns (no certificate).
4. Guards against duplicates: if a certificate already exists for `(UserId, LearningPathId)` → returns.
5. Otherwise creates a `Certificate` with `CertificateUrl = $"/certificates/{userId}/{pathId}"` (a stub URL), `IssuedAt = DateTime.UtcNow`.
6. Logs `CERTIFICATE_GENERATED` audit.

Certificate Listing

`CertificateController.GetMyCertificates()` returns the current user's certificates, ordered by `IssuedAt` descending, joined with the learning path title.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/certificates/CertificatesPage.tsx</code>	Card list of earned certificates
Service	<code>frontend/src/services/progressService.ts</code>	<code>markComplete</code> , <code>getMyProgress</code> , <code>getPathProgress</code>
Service	<code>frontend/src/services/certificateService.ts</code>	<code>getMy certificates</code>

Layer	File Path	Responsibility
Integration	frontend/src/pages/learningPaths/LearningPathDetailPage.tsx	Shows progress % and "Mark Complete" button (admin)

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/ProgressController.cs	POST complete, GET (my progress), GET paths/{pathId}
Controller	backend/Controllers/CertificateController.cs	GET /api/v1/certificates (my certificates)
Service	backend/Services/Progress/ProgressService.cs	Complete marking, summaries, certificate issuance
DTO	backend/DTOs/Progress/ProgressDto.cs	MarkCompleteDto, ProgressResponseDto, PathProgressSummaryDto, ModuleProgressDto
Interface	backend/Interfaces/Services/IProgressService.cs	Service contract

8. Database Implementation

Entity/Table	Purpose	Important Relationship
Progresses	Per-user per-module completion record	FK UserId → Users, FK ModuleId → Modules; IsCompleted, CompletedAt
Certificates	Issued completion certificates	FK UserId → Users, FK LearningPathId → LearningPaths; CertificateUrl, IssuedAt

- One `Progress` row exists per (user, module). Creating is idempotent via `upsert`.
 - A duplicate certificate for the same (user, path) is prevented by the `alreadyIssued` guard.
-

9. Security & Authorization

- All progress/certificate endpoints require authentication (`[Authorize]`).
 - `POST /progress/complete` requires `Instructor` or `Admin` role (students cannot self-approve module completion manually; quiz-pass automatically completes modules).
 - All lookups are scoped to the authenticated user id from the JWT (`ClaimTypes.NameIdentifier`) — a user can only see their own progress and certificates.
-

10. Important Business Rules

1. **Prerequisite enforcement:** A module whose prerequisites are incomplete cannot be marked complete.
2. **No re-completion:** An already-completed module cannot be marked complete again (`"Module already completed."`).
3. **Progress percent:** `round(completed/total * 100)`; certificate eligibility only at exactly 100%.
4. **Auto-issuance:** Certificates are issued only when the user has completed every module of the path, automatically, and only once per (user, path).
5. **Quiz pass integration:** Passing an assigned quiz automatically completes the module and may trigger issuance.
6. **Certificate URL stub:** `CertificateUrl` is a placeholder path (`/certificates/{userId}/{pathId}`); no actual certificate document/renderer exists.

Status: Partially implemented. Progress tracking (marking, summaries, unlock state, percentages) is fully functional. Certificate issuance produces database records only — the `CertificateUrl` is a stub and there is no real certificate document/PDF generation.

11. Example of Internal Execution

Step 1: Student completes the last module of a path

- **User Action:** Student passes the last quiz; `AttemptService` calls `MarkModuleCompleteFromQuizAsync(userId, moduleId)`.

- **Service:** The module is upserted to completed in `Progresses`; `MODULE_COMPLETED` audit logged.
- **Issuance:** `TryIssueCertificateAsync` counts completed progress for the path → equals total modules → no existing certificate → a `Certificates` row is created with stub `CertificateUrl` and `CERTIFICATE_GENERATED` audit logged.

Step 2: Student opens the Certificates page

- `certificateService.getMy()` → `GET /api/v1/certificates`.
- Backend returns the list with `LearningPathTitle` and `IssuedAt`; frontend renders certificate cards.

Step 3: Progress viewed in dashboard

- `GET /api/v1/progress` returns `PathProgressSummaryDto` (100%, `IsCertificateEligible = true`), which the dashboard renders as a completed progress bar.

Step 4: Prerequisite block

- An admin tries to mark Module 5 complete, but Module 3 (a prerequisite) is incomplete.
- `MarkCompleteAsync` counts completed deps → fewer than required → `ArgumentException` → HTTP 400 with "Complete all prerequisite modules first."

12. Files Involved

Frontend

- `frontend/src/pages/certificates/CertificatesPage.tsx`
- `frontend/src/services/progressService.ts`
- `frontend/src/services/certificateService.ts`
- `frontend/src/pages/learningPaths/LearningPathDetailPage.tsx`
- `frontend/src/pages/dashboard/DashboardPage.tsx` (progress overview)

Backend

- `backend/Controllers/ProgressController.cs`
- `backend/Controllers/CertificateController.cs`
- `backend/Services/Progress/ProgressService.cs`
- `backend/DTOs/Progress/ProgressDto.cs`
- `backend/Interfaces/Services/IProgressService.cs`

Database

- `backend/Entities/Progress.cs`
 - `backend/Entities/Certificate.cs`
-

13. Functionality

- Mark modules complete (with prerequisite enforcement)
- Auto-complete modules on quiz pass
- Per-user progress summaries per path (completed, total, percent, unlock state)
- Certificate eligibility detection (100%)
- Automatic one-time certificate issuance per completed path
- List own certificates
- Audit logging of module completion and certificate generation

Note: certificate document/PDF generation is **not implemented** — only certificate records exist.